Competitive Programming

A Deep Dive, from zero prior knowledge

Explainer for Salman Adnan's portfolio

The whole project in four sentences

This repository holds 361 C++ programs Salman wrote while training for programming contests, each one a solution to a puzzle from a contest website, filed by the technique it practises. A further 30 files that he studied but did not write sit apart in their own labelled folder and are never counted as his. Two small Python tools make the collection checkable rather than just browsable: one compiles every file, and one feeds the eight data-structure solutions thousands of random inputs and compares every answer with a slow version that is obviously right. A third checks every solution that was fixed after a line-by-line review against its own brute force.

Contents

1 Orientation: what this thing actually is **3**
- 1.1 The vocabulary you need first . . . 3
- 1.2 What the repository is for . . . 4

2 The shape of the repository **4**

3 How one solution file is built **5**
- 3.1 Anatomy . . . 5
- 3.2 The templates, and which files use which . . . 6
- 3.3 Reading and printing: `re` and `ps` . . . 6
- 3.4 The `LOCAL == SALMAN` guard . . . 7

4 The full template, section by section **7**

5 Range queries: the core of the repository **9**
- 5.1 Prefix sums, XOR and 2D . . . 9
- 5.2 Fenwick tree . . . 10
- 5.3 Segment tree . . . 10
- 5.4 Finding a position: Hotel Queries . . . 11
- 5.5 Lazy propagation . . . 12
- 5.6 Two tags: Range Updates and Sums . . . 12
- 5.7 Tags that are not constants: Polynomial Queries . . . 13
- 5.8 Sparse table: the one in the graphs folder . . . 13

6 How the repository checks itself **14**
- 6.1 `tools/build_all.py`: does every file compile? . . . 14
- 6.2 `stress/stress.py`: are the answers right? . . . 15
- 6.3 `stress/regressions.py`: do the fixes hold? . . . 15
- 6.4 The published run . . . 16
- 6.5 Checks that are not in the repository . . . 16

7 The collection, folder by folder **17**

8 Code that is not Salman's **18**

9 Honest assessment **18**
9.1 Bugs found by reading every file, and fixed . . . 18
9.2 Weak spots across the collection . . . 19
9.3 What is genuinely good . . . 20
9.4 What to do differently, starting over . . . 20

10 The story, briefly **20**

11 What to say about this project **21**

A Every file **21**
`practice/range-queries/` . . . 21
`practice/dynamic-programming/` . . . 21
`practice/graphs/` . . . 22
`practice/number-theory/` . . . 24
`practice/bitmasks/` . . . 25
`practice/prefix-sums/` . . . 26
`practice/two-pointers/` . . . 26
`practice/mixed/` . . . 27
`practice/cs-academy/` . . . 30
`practice/atcoder/` . . . 30
`contests/codeforces/` . . . 30
`contests/atcoder/` . . . 31
`course-cs334/class-contests/` . . . 31
`course-cs334/labs/` . . . 31
`course-cs334/problem-sets/` . . . 31
`from-other-sources/cses-dp-priyansh-agarwal/` . . . 31
`from-other-sources/contest-problems/` . . . 32

1 Orientation: what this thing actually is

1.1 The vocabulary you need first

The repository belongs to a hobby with its own dense vocabulary. Nothing below is assumed.

Jargon: Competitive programming

A sport in which people solve small, self-contained algorithm puzzles against a clock. You read a problem statement, write a program, submit it, and an automated judge decides whether it is right. Contests last from about an hour and a half to five hours.

Jargon: Online judge

The website that runs your program on hidden test inputs and compares its output with the expected output. The program must also finish within a time limit (typically one or two seconds) and a memory limit (typically 256 megabytes).

Jargon: Codeforces, CSES, AtCoder, CS Academy, USACO

The sites these problems come from. Codeforces runs frequent *rounds*, graded by *division* (Div. 4 is the easiest, Div. 1 the hardest). CSES is a fixed list of 300 classic training problems grouped by topic, including a "Range Queries" section that this repository treats in depth. AtCoder is a Japanese contest site with Beginner (ABC) and Regular (ARC) contests. CS Academy is an older contest site. USACO is the USA Computing Olympiad, whose problems read from and write to named files instead of the keyboard and screen.

Jargon: Standard input and output

A contest program does not open windows or files. It reads text from *standard input* (what you would type at the keyboard) and writes text to *standard output* (the screen). The judge pipes its test file into the first and captures the second. Every solution here is a program of exactly that shape.

Jargon: Big-O notation

A way of saying how a program's running time grows with the size of its input, n. $O(n)$ means "roughly proportional to n"; $O(\log n)$ grows very slowly (about 18 steps for $n = 200{,}000$); $O(n^2)$ is fine for $n = 2{,}000$ and hopeless for $n = 200{,}000$. Contest problems are designed so that only an algorithm with the right growth rate fits in the time limit, which is what makes the data structures in Section 5 worth learning.

Jargon: Brute force

The slow, obvious way to compute an answer: try everything, loop over every element. It is usually far too slow to submit, but it is easy to get right, which makes it the perfect reference to check a fast solution against.

Jargon: Stress testing

Generating many small random inputs, running both the fast solution and the brute force on each, and stopping at the first input where they disagree. The disagreeing input is a ready-made bug report. Section 6 describes the one in this repository.

Jargon: Upsolving

Solving a contest problem after the contest is over, usually after reading the official explanation (the *editorial*) or other people's code. It is how most competitors learn, and it matters here because it is also how other people's code ended up in the original folder (Section 8).

1.2 What the repository is for

Salman spent about eight months on competitive programming full time, starting with CS 334 (competitive programming) at Habib University and carrying on after it. That figure is his own: the file timestamps run from April to November 2025, but 185 of the 361 files carry the same date, 7 April 2025, which is what a bulk copy leaves behind, so the timestamps cannot date anything earlier than that.

The work lived in one folder with no version control, full of duplicates, placeholders and half-finished attempts. The repository is that folder cleaned into something a reader can browse and a machine can check. It is a record of practice, not a library: no file is imported by any other, and there is no program here that you "run" as a whole.

2 The shape of the repository

```
competitive-programming/
├── README.md   what is here and how it was checked
├── LICENSE   PolyForm Strict 1.0.0
├── .gitignore
├── template/
│   └── main-cpp-template.cpp   the full template, 639 lines
├── tools/
│   └── build_all.py   compile everything, 51 lines
├── stress/
│   └── stress.py   random inputs against brute force, 260 lines
└── solutions/
    ├── practice/   297 files, by topic
    ├── contests/   41 files from 12 live contests
    ├── course-cs334/   23 files from the university course
    └── from-other-sources/   30 files by other people, plus a README
```

Figure 1: Every tracked file outside `solutions/`, and the four groups inside it. `git ls-files` lists 398 files: 391 `.cpp` solutions and these seven.

Folder	Files	What is in it
`practice/range-queries`	8	the CSES Range Queries section, all stress-tested
`practice/dynamic-programming`	36	AtCoder DP contest, CSES DP, Codeforces DP
`practice/graphs`	65	traversal, trees, LCA, DSU; 15 build no graph at all
`practice/number-theory`	48	divisors, primes, gcd, modular arithmetic, probability
`practice/bitmasks`	8	bitwise tricks
`practice/prefix-sums`	13	prefix sums and difference arrays
`practice/two-pointers`	7	windows and pointer sweeps
`practice/mixed`	101	mixed-topic sets, the largest folder
`practice/cs-academy`	8	CS Academy problems
`practice/atcoder`	3	AtCoder practice
`contests/codeforces`	30	7 rounds, from Div. 4 to a combined Div. 1 + 2
`contests/atcoder`	11	5 Beginner and Regular contests
`course-cs334/class-contests`	8	in-class contests
`course-cs334/labs`	7	lab problems
`course-cs334/problem-sets`	8	graded problem sets
Salman's own	**361**	101,387 lines
`from-other-sources`	30	studied, not written: 1,557 lines

Table 1: Counts measured on the final tree. A file name follows its problem: `problem-27-ring-road.cpp` is the 27th problem of that topic set, Codeforces' "Ring Road". Gaps in the numbering are problems that were never solved, were duplicates, or were not kept (Section 10).

Folders are by intent, not by technique

The folder names come from the training sets Salman was working through, and a set's name says what it was meant to practise, not what each answer turned out to need. Fifteen of the 65 files in `graphs/` build no adjacency list or visited array at all (they are case analysis, string scans or simulations), four more build one and never traverse it, and one of them, `problem-59-static-range-minimum-queries.cpp`, is a sparse table, explained in Section 5.8. Three files in `number-theory/` (38, 46 and 47) are dynamic programming, and three in `prefix-sums/` (b, c and h) use no prefix sum. The folders are a fair map of what he studied, and a loose map of what each file does.

3 How one solution file is built

3.1 Anatomy

Most files are one program in three layers: a block of shared helpers pasted at the top, a `solve()` function written for this problem under a banner that reads `SOLVE`, and a `main()` that calls it.

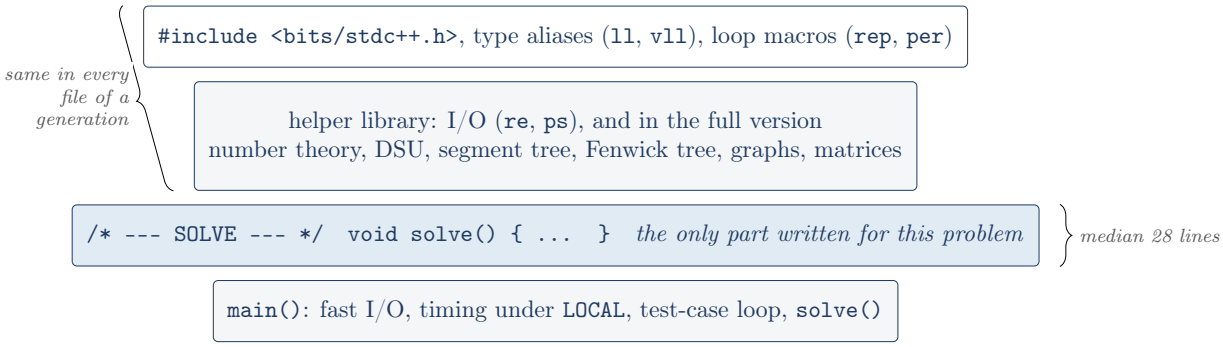


Figure 2: The layers of a template-based file. Across the 267 template-based files the `solve()` sections add up to 8,928 lines; everything else is pasted helper code.

How to read any file

Open it, search for `SOLVE`, and start reading there. Everything above that banner is the shared template; everything below is the answer to this file's problem.

3.2 The templates, and which files use which

Measured over all 361 files:

Template	Files	Where it is used
Full library (639 lines, or a 300 to 500 line cut of it)	105	mostly graphs, DP, number theory
Trimmed version (about 85 lines above `SOLVE`)	162	all range queries, the mixed set, most contest rounds
Older template with a `Solve Function` banner (230 to 330 lines)	19	number theory 13 to 32
Short template that reads `input.txt` under `-DLOCAL`	8	bitmasks, two pointers
No template	67	mostly CS 334 and early prefix-sum work

Table 2: Template use. Every file was matched by its banner text, not by eye.

The full library is the copy in `template/main-cpp-template.cpp`, last saved on 23 April 2025. The trimmed version is not in the repository as a separate file, but 68 files carry it byte for byte and the rest of the 162 are at least 90% similar; it keeps only the type aliases, the loop macros, `gcd` and `lcm`, the `re`/`ps` I/O functions, two debug macros and a random-number helper.

The template got smaller, not bigger

It is tempting to read the 639-line file as the final, most evolved template. The file dates say otherwise: no file that carries the full library is dated later than 11 April 2025, and every file that carries the trimmed version is dated 13 April 2025 or later, running to November. The full library came first, and the later work dropped almost all of it. The README was wrong about this until 26 October 2026 and has been corrected.

3.3 Reading and printing: `re` and `ps`

Both templates replace `cin » a » b` and `cout « x « "\n"` with two variadic helpers, which accept any number of arguments of any type:

```
template<class Arg,class...Args> void re(Arg &f,Args &...r){ re(f); re(r...); }
template<class T> void re(vector<T> &a){ for(auto &x:a) re(x); }
template<class Arg,class...Args> void ps(const Arg &f,const Args &...r){ pr(f); pr(" "); ps(r
    ...); }
```

Jargon: Variadic template

A C++ function that takes "any number of arguments". `re(N, Q)` reads two values, `re(arr)` reads a whole vector, and `re(N, Q, arr)` does both, because the function peels off the first argument, reads it, and calls itself on the rest.

So `ll N, Q; re(N, Q); vll arr(N); re(arr);` reads a whole input header and array in one line, and `ps(a, b)` prints `a b` and a newline.

3.4 The `LOCAL == SALMAN` guard

Most template-based files (227 of the 267) wrap their timing code in this guard, and 40 files put a redirect that reads input from a local file behind it or a plain `LOCAL` check:

```
#if defined(LOCAL) && LOCAL == SALMAN
    auto start = std::chrono::high_resolution_clock::now();
#endif
```

Jargon: Preprocessor and `-D`

Before C++ is compiled, a *preprocessor* decides which lines exist at all. Compiling with `-DLOCAL` defines the name `LOCAL`; `#if` keeps or drops code depending on such names. An undefined name inside `#if` counts as 0.

The guard is subtler than it looks. `SALMAN` is never defined anywhere, so inside `#if` it counts as 0. Plain `-DLOCAL` defines `LOCAL` as 1, and $1 = 0$ is false, so the block stays off. It switches on only for `-DLOCAL=SALMAN` (which makes `LOCAL` expand to `SALMAN`, hence $0 = 0$) or for defining both names. This was checked with the real preprocessor:

Compiled with	Timing block
nothing (what a judge does)	off
`-DLOCAL`	off
`-DLOCAL=SALMAN`	on
`-DLOCAL -DSALMAN`	on

Table 3: Output of `cpp -P` on the guard for each set of flags.

On a judge none of it runs, which is the point: the timing line would otherwise be printed into the answer and make every answer wrong.

4 The full template, section by section

`template/main-cpp-template.cpp` is 639 lines in 17 numbered sections. It compiles on its own (it is part of the build check) and does nothing but read one number.

#	Section	What it provides
	header	`bits/stdc++.h` (the whole standard library in one include), fast I/O macro, type aliases `ll` (64-bit integer), `vll`, `pll` and friends, loop macros `rep`/`per`, `MOD` $= 10^9 + 7$, `INF`
1	short helpers	`gcd`, `lcm`, a dozen bit tricks (`is_power_of_two`, popcount, lowest set bit, XOR of 1 to n)
2	number theory	`binpow`, `binpowmod` (fast exponentiation), modular inverse by Fermat and by extended Euclid, trial-division primality, divisor count
3	linear sieve	primes and smallest prime factor up to 10^6; `prime_factors`
4	factorials	exact factorial (with an overflow check) and Stirling's approximation
5	DSU	disjoint set union with path compression and union by size
6	segment tree	lazy range add, range sum (the same structure as Section 5.5)
7	more math	Euler's totient up to 10^6, Chinese remainder theorem
8	modular arithmetic	a `modnum<M>` struct, `mul`, `add`, `divide`, factorial tables and four ways to compute $\binom{n}{r}$
9	Möbius function	up to 10^6
10	Fenwick tree	1-indexed point add, prefix and range sum (Section 5.2)
11	discrete log	baby-step giant-step: solve $a^x \equiv b \pmod m$
12	matrices	multiplication and fast power, for linear recurrences
13	graphs	BFS, iterative DFS, Kahn's topological sort, LCA by binary lifting, Tarjan and Kosaraju strongly connected components
14	fast I/O	the `re`/`ps` helpers above
15	random	a 64-bit Mersenne Twister seeded from the clock
16	SOLVE	an empty `solve()` that reads one number
17	MAIN	fast I/O, guarded timing, a test-case loop (multi-test commented out)

Table 4: The 17 sections of the full template.

Jargon: Modular arithmetic and $10^9 + 7$

Many problems ask for an answer that would have hundreds of digits, so they ask for it "modulo $10^9 + 7$", the remainder after dividing by that prime. Doing every addition and multiplication modulo a prime keeps numbers small, and because the modulus is prime, division also works, by multiplying with a *modular inverse*.

Jargon: Disjoint set union (DSU)

A structure that tracks which items are in the same group and can merge two groups, both in nearly constant time. Graph problems use it to answer "are these two points connected yet?" while edges are being added.

What was wrong with the full template, and what was fixed

Read line by line, the template had defects that never bit only because the affected functions were never called by any solution. They were fixed on 26 October 2026 in `template/main-cpp-template.cpp` and are checked by `stress/regressions.py` (Section 6.3):

- **Declarations that matched nothing.** Line 41 forward-declared `count_divisors`,

`mat_mul`, `mat_pow` and a zero-argument `precompute_factorials()`, but the definitions are `countDivisors`, `math_mul`, `math_pow` and take an argument. The declarations now match.

- **CRT assumed prime moduli.** It inverted with Fermat's little theorem, which is only correct for a prime modulus. It now uses the extended-Euclid inverse and 128-bit intermediate products, so any pairwise-coprime moduli work.
- **The discrete log was off by one** (for $2^x \equiv 2 \pmod 5$ it returned 0, but $2^0 = 1$), kept the largest exponent for a repeated value instead of the smallest, and also inverted with Fermat. All three are fixed.
- **Overflow detection by overflow.** `factorial` checked `r < 0` after multiplying, but signed overflow is undefined behaviour in C++. It now uses `__builtin_mul_overflow` and returns -1 past 20!, and `compute_factorial` no longer casts Stirling's approximation into a 64-bit integer.
- `INF = LLONG_MAX` is unchanged: adding anything to it overflows, so it is only safe as a value that is compared, never summed.
- **Memory paid up front** is unchanged too. Five global tables of a million 64-bit integers are allocated whether or not they are used: the full template takes 41 MB just to read one number, against 7 MB for a trimmed-template solution.

The 105 solutions that paste the full template keep their own copies of the old helpers. None of them calls the affected functions, so their behaviour does not depend on it.

5 Range queries: the core of the repository

The eight files in `practice/range-queries/` are the CSES Range Queries section, and they are where the data structures live. Each problem gives an array of up to 200,000 numbers and up to 200,000 questions about ranges of it, sometimes mixed with updates. A brute-force loop costs up to $200{,}000 \times 200{,}000 = 4 \times 10^{10}$ steps, far past any time limit; every structure below brings a query down to $O(\log n)$ or $O(1)$.

Problem	Structure	Cost per operation
Range Xor Queries	prefix XOR	$O(1)$ query
Forest Queries	2D prefix sums	$O(1)$ query
Dynamic Range Sum Queries	Fenwick tree	$O(\log n)$
Dynamic Range Minimum Queries	segment tree	$O(\log n)$
Hotel Queries	max segment tree with descent	$O(\log n)$
Range Update Queries	lazy segment tree (add)	$O(\log n)$
Range Updates and Sums	lazy segment tree (add and assign)	$O(\log n)$
Polynomial Queries	lazy segment tree (progressions)	$O(\log n)$

Table 5: The eight range-query solutions. All are stress-tested (Section 6).

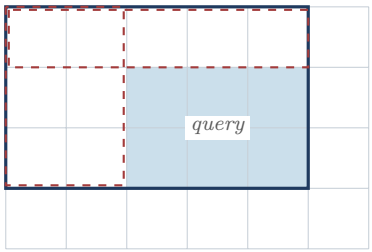
5.1 Prefix sums, XOR and 2D

Jargon: Prefix sum

An array p where $p[i]$ is the sum of the first i elements. The sum of any range $[a, b]$ is then $p[b] - p[a-1]$: two lookups instead of a loop. It only works while the array never changes.

Range Xor Queries uses the same idea with XOR instead of addition. XOR is its own inverse ($x \oplus x = 0$), so `prf_xor[b] ^ prf_xor[a-1]` cancels the shared prefix exactly as subtraction does.

Forest Queries counts trees in rectangles of an $n \times n$ grid. The solution builds a 2D prefix table where `dp[i][j]` is the number of trees in the rectangle from the top-left corner to (i, j), and answers a query with inclusion-exclusion:



answer = `dp[y2][x2]` − `dp[y1-1][x2]`
− `dp[y2][x1-1]` + `dp[y1-1][x1-1]`

the corner strip is subtracted twice,
so it is added back once

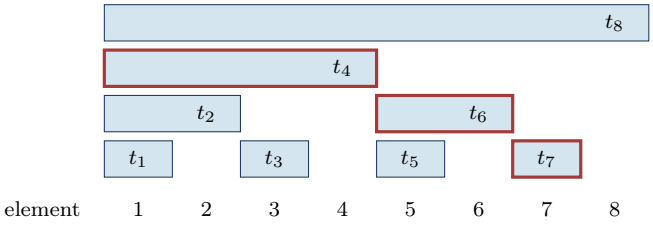
Figure 3: 2D inclusion-exclusion: the big rectangle, minus the strip above, minus the strip to the left, plus the corner that was removed twice.

5.2 Fenwick tree

Jargon: Fenwick tree (binary indexed tree)

An array where each slot holds the sum of a block of the original array, with block lengths chosen from the binary representation of the slot's index. Both "add d to element i" and "sum of the first i elements" touch only $O(\log n)$ slots.

Slot i (counting from 1) covers the lowbit(i) elements ending at i, where lowbit(i) is the value of i's lowest set bit, written `i & -i` in C++.



sum of $1..7 = t_7 + t_6 + t_4$
$7 \to 6 \to 4 \to 0$: each step removes the lowest set bit

Figure 4: What each slot of an 8-element Fenwick tree covers. A prefix query walks down by removing the lowest bit; an update walks up by adding it.

```
1 inline void update(ll i,ll d){ for(;i<=n;i+=(i&-i)) fenw[i]+=d; }
2 inline ll query(ll i){ ll s=0; for(;i>0;i-=(i&-i)) s+=fenw[i]; return s; }
```

Dynamic Range Sum Queries must *assign* a value, but a Fenwick tree only knows how to add. The solution keeps a plain copy of the array beside the tree and turns "set position k to u" into "add u − old at k".

5.3 Segment tree

Jargon: Segment tree

A binary tree over the array. The root covers the whole array, each node's two children cover its left and right halves, and each leaf is one element. Every node stores the answer for its range (a sum, a minimum, a maximum). Any range splits into at most about $2 \log_2 n$ whole nodes, so a query combines a handful of stored answers instead of scanning elements.

The solutions store the tree in a flat array with *heap indexing*: the root is node 1, and node i's children are $2i$ and $2i + 1$. Arrays are sized $4n$, a standard safe bound.

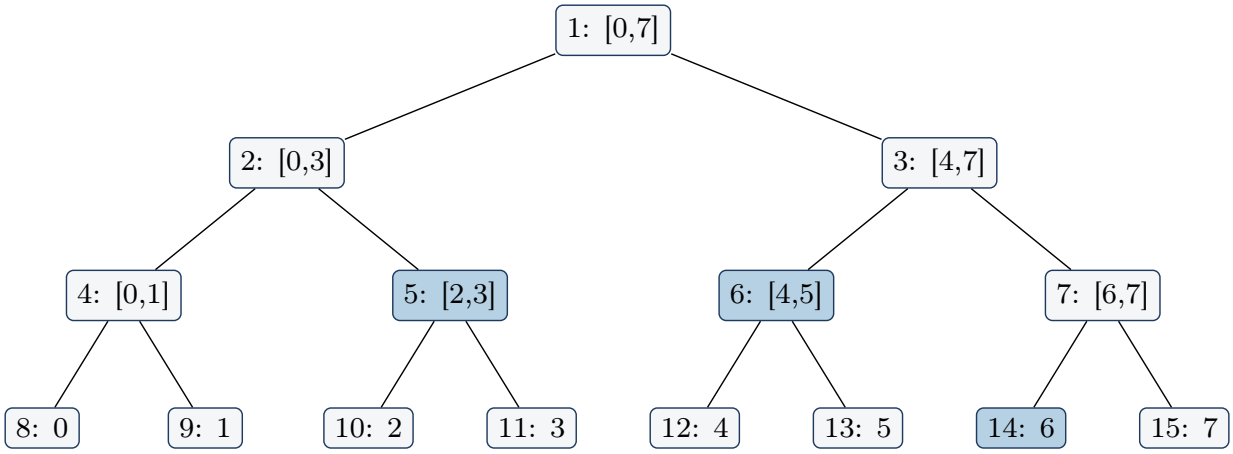


Figure 5: Heap-indexed segment tree over 8 elements. The query $[2, 6]$ is answered by combining exactly the three shaded nodes, 5, 6 and 14.

Dynamic Range Minimum Queries stores minimums. A query that lands wholly outside a node returns `INF`, one wholly inside returns the stored minimum, and anything else recurses into both children:

```
ll query_internal(ll i, ll l, ll r, ll ql, ll qr){
    if(ql>r||qr<l) return INF;
    if(ql<=l&&r<=qr) return tree[i];
    ll m=(l+r)/2;
    return min(query_internal(i*2,l,m,ql,qr),query_internal(i*2+1,m+1,r,ql,qr));
}
```

A build function that is never called

The minimum tree has an $O(n)$ `build()`, but `solve()` fills the tree with n separate point updates, $O(n \log n)$. The answer is the same and it is fast enough, but the better function sits unused right above it.

5.4 Finding a position: Hotel Queries

Each group of tourists takes the *first* hotel with enough free rooms. The tree stores the maximum free rooms in each range, and instead of binary searching (which would cost $O(\log^2 n)$), the query walks down from the root: if the left child's maximum is big enough, the answer is on the left; otherwise it is on the right.

```
int query(int node, int l, int r, ll x) {
    if (tree[node] < x) return -1;          // no hotel in this range is big enough
    if (l == r) return l;                   // reached the hotel
    int mid = (l + r) / 2;
    if (tree[2 * node] >= x) return query(2 * node, l, mid, x);
    return query(2 * node + 1, mid + 1, r, x);
}
```

One root-to-leaf walk, $O(\log n)$, then a point update takes the rooms. The idea worth remembering is that a stored aggregate can answer "where", not just "how much".

5.5 Lazy propagation

Jargon: Lazy propagation

Adding a value to every element of a range would touch up to n leaves. Instead, when an update fully covers a node, the tree updates that node's stored sum and leaves a note, a *lazy tag*, saying "my children still owe this addition". The note is only pushed down to the children when a later operation needs to look inside the node.

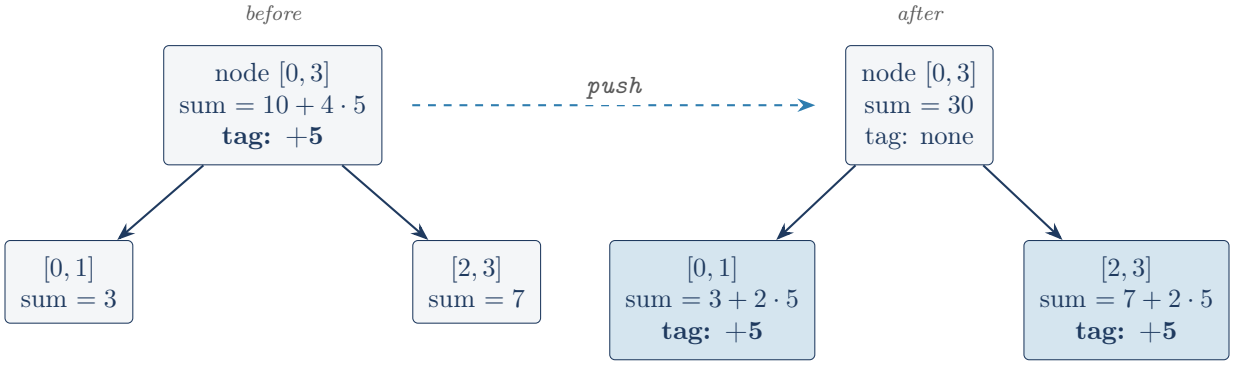


Figure 6: Before and after `push`: the parent's tag moves to both children, each child adds tag × its length to its sum, and the parent's tag is cleared.

```
inline void apply(ll idx,ll l,ll r,ll val){ tree[idx]+=val*(r-l+1); lazy[idx]+=val; }
inline void push(ll idx,ll l,ll r){
    if(lazy[idx]){ ll m=(l+r)/2;
        apply(idx*2,l,m,lazy[idx]); apply(idx*2+1,m+1,r,lazy[idx]); lazy[idx]=0; }
}
```

Range Update Queries uses exactly this tree for "add u to $[a, b]$" and "read element k" (a query on $[k, k]$).

More machinery than the problem needs

Range add with single-element reads does not need a lazy tree. A Fenwick tree over the *difference array* (add u at a, subtract it at $b + 1$, read element k as a prefix sum) does the same job in about a third of the code. The README says so in "What I'd do differently".

5.6 Two tags: Range Updates and Sums

This problem mixes "add x to $[a, b]$" with "set $[a, b]$ to x", so a node can owe its children either kind of change, and the order matters: an add that arrives after a set must change the set value, and a set that arrives after an add must wipe the add. The node keeps two tags, `lazy` (a pending add) and `is_set` (a pending assignment, -1 meaning none), and `apply` folds them so that at most one is ever pending:

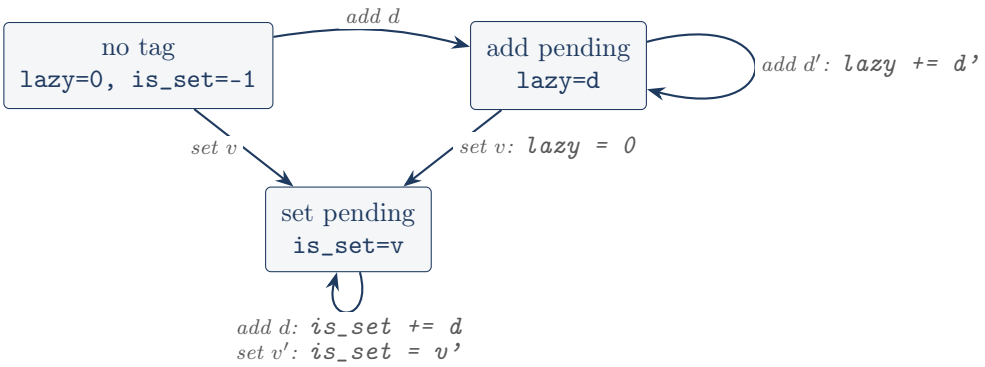


Figure 7: The tag states of one node. An add on a pending set is folded into the set value instead of being stacked, so `push` only ever has one kind of tag to hand down.

A sentinel that leans on the input limits

-1 means "no assignment pending". That is safe only because CSES guarantees every value is at least 1 and every add is positive, so no real assignment can ever equal -1. Change the constraints and this tree silently breaks; a separate boolean flag would not.

5.7 Tags that are not constants: Polynomial Queries

Here an update adds $1, 2, 3, \ldots$ across $[a, b]$: the first element gets 1, the second 2, and so on. The pending change on a node is no longer one number, it is a line. The solution stores it in absolute index terms, as two numbers c and d meaning "element i gains $c + d \cdot i$". For an update starting at L (0-indexed), element i must gain $i - L + 1$, so $c = 1 - L$ and $d = 1$.

Two pending lines add coefficient by coefficient, so tags still combine by addition. And a node covering $[l, r]$ of length len gains

$$\sum_{i=l}^{r} (c + d\,i) \;=\; c \cdot len + d \cdot \frac{(l + r)\,len}{2},$$

which is exactly what `apply` computes:

```
inline void apply(ll idx, ll l, ll r, ll c, ll d) {
    ll len = r - l + 1;
    ll sum_i = (l + r) * len / 2;          // sum of the indices l..r
    tree[idx] += c * len + d * sum_i;
    lazy_c[idx] += c; lazy_d[idx] += d;
}
```

The lesson of the three lazy trees

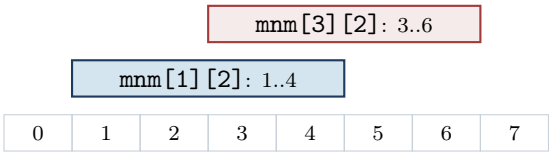
Every bug in a lazy segment tree is a disagreement about what a pending tag *means*. Writing that meaning down first ("a pending add", "a pending assignment that absorbs later adds", "a line $c + d\,i$ in absolute indices") is what made each of these short.

5.8 Sparse table: the one in the graphs folder

`practice/graphs/problem-59-static-range-minimum-queries.cpp` solves CSES Static Range Minimum Queries with a *sparse table*. It is filed under graphs only because that is the training set it came from.

Jargon: Sparse table

A table where `mnm[i][k]` holds the minimum of the 2^k elements starting at i. Row k is built from row $k-1$ by combining two halves. A query on $[L, R]$ picks the largest 2^k that fits and takes the minimum of two blocks that start at L and end at R; they may overlap, which is harmless for a minimum. Build $O(n \log n)$, query $O(1)$, but no updates.



query $[1, 6]$: length 6,
$k = \lfloor \log_2 6 \rfloor = 2$,
answer = min of two blocks of 4

Figure 8: An $O(1)$ range-minimum query from two overlapping power-of-two blocks.

```
ll query(ll L, ll R) {
    ll len = R - L + 1;
    ll k = (ll)log2(len);
    return min(mnm[L][k], mnm[R-(1<<k)+1][k]);
}
```

A correction on record

When this repository was first published, Salman's old resume listed sparse tables and the project notes said no file implemented one. That note was wrong: this file does. The resume line now includes sparse tables again. One small weakness in the code itself: `log2` is a floating-point function, and `1<<k` is a 32-bit shift; both are fine at these sizes, but an integer `__lg` would be the exact tool.

6 How the repository checks itself

6.1 `tools/build_all.py`: does every file compile?

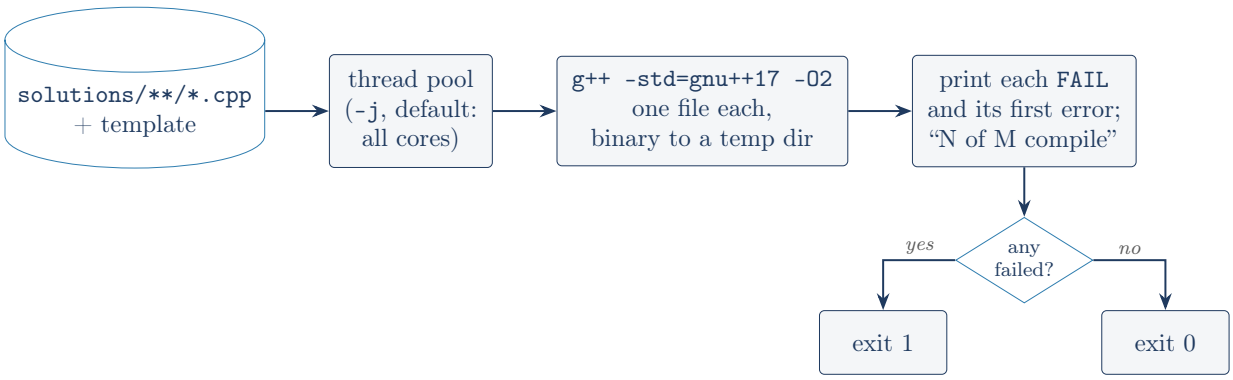


Figure 9: The build check. Binaries are thrown away; the only question is whether every file compiles with the flags a judge uses.

Jargon: `-std=gnu++17 -O2`

Compiler flags: compile as C++17 with GNU extensions, and optimise. These are the settings most judges use, so "compiles here" means "would compile there".

The script reads `CXX` from the environment, so any compiler that accepts g++ flags can stand in. It does not run anything.

6.2 `stress/stress.py`: are the answers right?

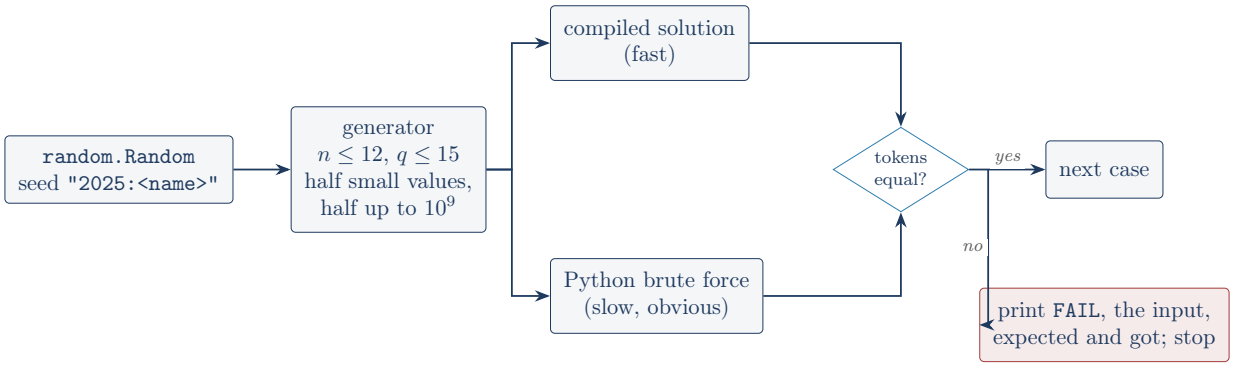


Figure 10: One stress case. By default 500 cases per problem; the published run used 1,000.

For each of the eight range-query problems the script has a *generator* that writes a random input in the problem's exact format and a *brute force* that answers it with plain loops. Three design choices make it trustworthy:

- **Small inputs, nasty values.** Arrays of at most 12 elements keep every case instant and every failure readable, but half the time values go up to 10^9, so a solution that overflowed a 32-bit integer would be caught.
- **A fixed seed per problem** (`f"{seed}:{name}"`), so a failure reproduces exactly, and running one problem with `--only` gives the same cases as running all.
- **Token comparison.** Output is split on whitespace before comparing, so a trailing space or a missing final newline is not mistaken for a wrong answer. Hotel Queries, for instance, prints its answers space-separated on one line, and the brute force returns a list; both reduce to the same tokens.

Proving the check can fail

A test that has never failed proves nothing. On 25 October 2026 the minimum-query solution was deliberately broken (`min` swapped for `max` in the tree's merge) and the stress test failed on its first case. The solution has not changed since.

6.3 `stress/regressions.py`: do the fixes hold?

Every solution fixed after review (Section 9.1) has its own test here, built the same way as the range-query stress test: a generator for small random inputs in the problem's format, and a brute force written from the problem statement. Three twists:

- **Checkers for problems with many right answers.** Carousel and AND Reconstruction accept any valid output, so their tests check the output instead of comparing it: the colouring must be valid and use the minimum number of colours, and the array must reproduce every given AND (or -1 must be printed exactly when no array exists, which the brute force decides by trying every small array).
- **Exhaustive search where a formula would be a guess.** Hamiiid, Haaamid... Hamid is a two-player game, so its test plays the whole game by minimax over every board state; IT Restaurants tries every assignment of the two chains to the vertices.

- **Sanitizers.** With `--sanitize`, every fixed solution is built with AddressSanitizer and UndefinedBehaviorSanitizer, so an out-of-bounds access or a signed overflow aborts the run instead of passing silently.

The same script compiles a small test program around `template/main-cpp-template.cpp` and checks its repaired helpers against brute force: the discrete log for every base and target under small moduli, CRT for random pairwise-coprime (and often non-prime) moduli, the factorials up to and past 20!, and the renamed declarations. On the old template the same 5,436 checks produce 2,756 failures; on the fixed one, none.

6.4 The published run

On 26 October 2026, in a clean `gcc:13` Docker container (g++ 13.5, Python 3.11), with the repository mounted read-only:

```
$ docker run --rm -v "$PWD":/r:ro -w /r gcc:13 sh -c \
    'python3 -B tools/build_all.py -j 8; echo BUILD_EXIT=$?;
     python3 -B stress/stress.py --cases 1000; echo STRESS_EXIT=$?'
392 of 392 files compile with g++ -std=gnu++17 -O2
BUILD_EXIT=0
ok   dynamic-range-sum-queries: 1000 random cases match brute force
ok   dynamic-range-minimum-queries: 1000 random cases match brute force
ok   range-xor-queries: 1000 random cases match brute force
ok   range-update-queries: 1000 random cases match brute force
ok   forest-queries: 1000 random cases match brute force
ok   hotel-queries: 1000 random cases match brute force
ok   range-updates-and-sums: 1000 random cases match brute force
ok   polynomial-queries: 1000 random cases match brute force
STRESS_EXIT=0
```

392 is Salman's 361 solutions, the 30 files from other sources, and the template. The regression tests, in the same container on the same day:

```
$ docker run --rm -v "$PWD":/r:ro -w /r gcc:13 sh -c \
    'python3 -B stress/regressions.py --sanitize; echo REGRESSIONS_EXIT=$?'
ok   it-restaurants: 300 cases
ok   hamid: 300 cases
ok   waste-sorting: 400 cases
ok   carousel: 300 cases
ok   sum-in-the-tree: 300 cases
ok   broken-keyboard: 400 cases
ok   static-range-sums: 400 cases
ok   and-reconstruction: 300 cases
ok   shohag-xor: 400 cases
ok   vlad-division: 400 cases
ok   infinite-sequence: 400 cases
ok   sakurako: 400 cases
ok   abc396-c: 400 cases
ok   least-prefix-sum: 400 cases
ok   star-sky: 400 cases
ok   arc194-a: 400 cases
ok   template-helpers: 5436 checks, 0 failures
REGRESSIONS_EXIT=0
```

6.5 Checks that are not in the repository

Three more sweeps were run once while the collection was curated. They are not scripts in the repository, so they are described here rather than claimed as features:

- **Does it print anything?** Every file was searched for an output statement in its `solve` or `main`. Three class-contest files read their input and never print an answer; they were removed.

- **Does it die before reading input?** Every solution was compiled and started with an input pipe that never sends data. A correct program waits; of the 362 files at the time, 360 did. One USACO solution (`problem-28-wormhole-sort.cpp`) exited because it reads a named file instead, which is correct for USACO. One Codeforces attempt crashed with a division by zero in a helper that `main` calls before reading anything; it was removed, and the later version of the same problem was kept.
- **Who wrote it?** Every file was searched for names, foreign templates and setups that were never Salman's. Section 8 has the result.

What these checks cover

Correct answers are machine-checked for the eight range-query solutions and for the 18 solutions fixed after review (Section 9.1). For the other 335, the machine evidence is that they compile and start without crashing. The bugs in Section 9.1 are exactly the kind a compile check cannot see, which is why they were hunted by reading every file.

7 The collection, folder by folder

Appendix A lists every one of the 391 files with its problem and the technique its code actually uses, read from each file's solve section. This section gives the shape of each folder.

`practice/graphs/` **(65)**
Mostly trees, traversed with recursive DFS more often than not, sometimes with an explicit stack or a BFS queue. Six files use binary lifting for the lowest common ancestor (LCA) and the k-th ancestor, sharing one layout. Four use a DSU, one uses the template's Kosaraju strongly-connected-components code, and several are bipartite two-colourings. Two are Euler-tour problems: one with a Fenwick tree (using the 0-indexed Fenwick variant that is commented out in the template), one with a lazy segment tree of colour bitmasks. Fifteen files build no graph structure at all.

`practice/dynamic-programming/` **(36)**
The AtCoder Educational DP contest (Frog 1 and 2, Vacation, both Knapsacks, LCS), Salman's own CSES DP solutions (Dice Combinations, Minimizing Coins, Coin Combinations I and II, Removing Digits, Grid Paths), and Codeforces DP problems. Two more CSES problems sit here where they were filed: Static Range Sum Queries and Increasing Subsequence.

`practice/number-theory/` **(48)**
Divisor enumeration, sieves, gcd and lcm identities, modular inverses, inclusion-exclusion counting, and a run of probability and expected-value problems (35, 37, 38, 39 and 44). Numbers 13 to 32, except 20, use the older `Solve Function` template.

`practice/mixed/` **(101)**
Mixed-topic sets, mostly Codeforces Div. 2 and 3 problems, plus six CS Academy problems. Mostly greedy, constructive and case-analysis answers, with DP, binary search and one segment tree (Limited Swaps) among them. One problem (88) is *interactive*: the program asks the judge questions and must flush its output after each one.

Prefix sums, bitmasks and two pointers (28)
Early practice: difference arrays, prefix XOR, sliding windows.

`contests/` **(41)**
Code written during live contests: Codeforces Rounds 1002, 1008, 1017, 1041, 1042, 1043 and 1065, and AtCoder ABC 396, 398 and 399 and ARC 194 and 195. ABC 399 F is kept in three versions, the second heavily commented.

`course-cs334/` **(23)**
Class contests, labs and problem sets from the university course, almost all written without any template in plain `int main()` style. They are very likely the earliest code in the collection (an inference: the course came first, and the timestamps cannot confirm it).

8 Code that is not Salman's

The original folder mixed Salman's work with code he had studied. A curation pass on 26 October 2026 searched every file for signs of another author and moved 30 files into `solutions/from-other-sources/`, which has its own README listing each one and why.

- **20 reference solutions headed** `/* Priyansh Agarwal*/`, a well-known competitive programming teacher, for the CSES DP problems. They came as one folder of downloaded code (all 20 share one timestamp, 18 November 2023) beside a folder of notes from YouTube. Salman's own solutions to the same problems are in `practice/dynamic-programming/`.
- **10 contest solutions in someone else's template or style**: one in galen_colin's public template (it carries his `galen_colin_local` guard), one in jiangly's (the `i64`/`u64`/`u128` aliases), one that includes a `templates/debug.h` and one that reads `../IO/c.in`, paths that exist nowhere in Salman's folders, one written like editorial code, three in styles unlike any of his, and two second versions of CS Academy problems whose comments read like chatbot output.

Salman's own account is that he edited many of these while learning from them, so they stay in the repository, clearly labelled, compiled with everything else, and excluded from every count. The decision on the borderline ones was his.

Why this section exists

When the repository was first published on 25 October 2026 these 30 files were counted among "398 of my solutions". That was wrong, and the count on the website was wrong with it, for about a day. Both now say 361. It is worth knowing because the question "did you write all of this?" deserves a precise answer, and now has one.

9 Honest assessment

9.1 Bugs found by reading every file, and fixed

Reading every solve section turned up bugs that a compile check cannot see. All of them were fixed on 26 October 2026, and each fix is tested by `stress/regressions.py` against a brute force written from the problem statement (Section 6.3). Running the same tests on the old versions is how the first group was confirmed: every one of those old versions crashed under AddressSanitizer or printed a wrong answer.

Real bugs (the old code failed its test):

- `mixed/problem-80-cs-academy-consecutive-subsequence.cpp`
  An input value of 0 read `best[-1]`. The index is now offset by two.
- `graphs/problem-5-carousel.cpp`
  The reset loop ran before `n` was read and wrote one element past the end.
- `graphs/problem-36-sum-in-the-tree.cpp`
  The root read `sm[-1]`; the parent value is now only read when there is a parent.
- `dynamic-programming/problem-29-static-range-sum-queries.cpp`
  The prefix array was one element short.

- `class-contests/contest-7/problem-b-and-reconstruction.cpp`
  `b[i] & b[i+1] != a[i]` parsed as `b[i] & (b[i+1] != a[i])`; it also printed `1` instead of `-1` and printed the answer with no spaces.
- `class-contests/contest-7/problem-c-shohag-loves-xor-easy-version.cpp`
  The counter was never initialised, `m` was read into an `int`, and the loop ran to `m`. No $y \geq 2x$ can qualify, so it now stops at $\min(m, 2x)$.
- `labs/lab-05/problem-d-and-an-infinite-sequence.cpp`
  The bit test `((1 << i) & 1) == 0` ignored the input. The answer is the OR of every value in $[n-m, n+m]$; bit i is set when the smallest value $\geq n-m$ with that bit is at most $n+m$.
- `labs/lab-04/problem-e-sakurakos-field-trip.cpp`
  The loop read `a[-1]` and visited each mirrored pair twice; it now visits each pair once.
- `contests/atcoder/2-beginner-contest-396/problem-c.cpp`
  Sorted ascending, then read the largest values from the front. Rewritten with descending prefix sums: the best total with k white balls and at least k black ones.
- `prefix-sums/problem-f-star-sky.cpp`
  Summed rows and columns separately, which cannot answer a rectangle query. Rewritten with 2D prefix sums kept per starting brightness.
- `contests/atcoder/3-regular-contest-194-div-2/problem-a.cpp`
  A hand-made greedy that could read an empty stack. A pop always removes the element pushed just before it, so the dropped steps form even-length blocks and $d_i = \max(d_{i-1}+a_i,\ d_{i-2})$ solves it.
- `graphs/problem-42-it-restaurants.cpp`
  Paired node degrees, which is not what the problem asks. The achievable pairs are the subset sums of the parts left when one vertex is removed; the fix computes them with a bitset for every vertex, iteratively, since a recursive version with a bitset in every frame overflows the stack on a 5,000-vertex path.

Latent bugs (wrong in the code, but the old version still passed its test): a `c4` that should have been `c5` in Waste Sorting (the capacity check before it already decides the answer); a 25-slot `vector<bool>` indexed at 25 in Yet Another Broken Keyboard (it lands inside the same 64-bit word); `(1 << 31) - 1` in Vlad and Division (signed overflow that GCC happens to fold correctly); a leftover cap of ten flips in Least Prefix Sum (provably never reached: one flip always covers the new element); a bounds check done after the read in Math; and a scan that stepped the wrong way in Hamiiid, Haaamid... Hamid (checked against a full minimax search of the game on 1,500 random boards, before and after). All six are fixed anyway.

9.2 Weak spots across the collection

- **Template weight.** 105 files paste a library of hundreds of lines to use a handful of functions. It buries the answer and costs 41 MB at startup (Section 4).
- **Floating point where integers belong.** Several files compute integer answers through `double` or `log2` (Cloudberry Jam, To Zero, the sparse table's `log2`). These work at contest sizes and are fragile in general.
- **Leftovers.** Commented-out earlier solutions (the biggest is about 150 lines in a CS 334 lab file), commented debug prints, unused variables and helpers. None of it prints on a judge: every live debug line is behind a `LOCAL` guard.
- **Naming.** Six files in `mixed/` have a number but no problem name (`problem-46.cpp` and so on), four numbers are each used for two files (51 and 102 in mixed, 13 in DP, 41 in number theory), and `understand.cpp` in DP is named after a mood, not a problem.

- **Large fixed allocations.** A few files allocate far more than they need (about 120 MB of prime factors in Enlarge GCD, a 5005×5005 table that is cleared three times per test in one of the files from other sources).

9.3 What is genuinely good

- The eight range-query solutions are clean, short and proven against brute force, and the three lazy trees show real understanding of what a tag means.
- The collection covers the standard contest toolkit end to end: prefix sums and difference arrays, Fenwick and segment trees (plain, lazy, descending), a sparse table, binary lifting, DSU, SCCs, Euler tours, DP from knapsack to digit and probability DP, sieves and modular combinatorics, and interactive problems.
- The repository is honest about itself: the README says exactly what is checked and how, and the curation history (duplicates, placeholders, non-solutions, other people's code, bugs found later) is written down rather than hidden.

9.4 What to do differently, starting over

- Write a brute force before trusting a fast solution. Every bug in Section 9.1 would have been caught by a two-minute brute force and a few hundred random cases, which is exactly how they were confirmed in the end.
- Keep helpers in one header instead of pasting them, and use a precompiled `bits/stdc++.h`; the full build of 392 files took several minutes.
- Prefer the simpler structure when it suffices (a Fenwick tree on the difference array for range add and point query) and a boolean flag over a sentinel value.
- Use version control from day one; the duplicates and backups that curation had to clear out are what an untracked folder accumulates.

10 The story, briefly

1. **The work.** About eight months of full-time practice by Salman's account, from CS 334 at Habib University onward. The course files are plain `int main()` C++ and are probably the earliest; the template-based files came later, first with the full library, then the trimmed one.
2. **First curation, 25 October 2026.** The original folder held 480 candidate `.cpp` files. Byte-identical copies (19), empty placeholders (17), editor backups (5) and three attempts that never compiled were left out; everything kept was compiled. A folder of ICPC 2026 solutions written in an assistant session that same day was excluded on purpose, as were contest folders whose authorship could not be confirmed.
3. **Second pass, 26 October 2026.** Reading every file found the template history above, 30 files by other people (moved, not deleted), and seven files that were not solutions at all: an empty template, a popcount experiment, an attempt still printing its debug arrays, three class-contest files that never print an answer, and an attempt that crashes before reading input (dropped). The count settled at 361.
4. **Third pass, later on 26 October 2026.** Every catalogue line in Appendix A was checked against its file, and the bugs in Section 9.1 were fixed, each with its own brute-force test (Section 6.3).

11 What to say about this project

- "It's eight months of contest practice in C++, cleaned into a repository a machine can check: every file compiles, and the data-structure solutions are stress-tested against brute force on a thousand random inputs each."
- On lazy propagation: be ready to explain the add-and-assign state diagram and the $c + d\,i$ tag of Polynomial Queries. Those two are the strongest evidence of understanding in the repository.
- On Hotel Queries: a segment tree can find *where*, not only *how much*.
- On honesty: "I only claim what I can re-check. Thirty files I studied but didn't write are kept separate, and when a full review found bugs, each fix came with a brute-force test that the old code fails."
- On weaknesses: the template weight, the sentinel in the two-tag tree, and the bugs in Section 9.1, which a brute force would have caught at the time. Owning them is better than being shown them.

A Every file

One line per file: its problem, and the technique its code implements, read from the code itself. The range-query files and the template are explained in depth in Sections 4 and 5; the files under `from-other-sources/` are listed for completeness and are not Salman's work.

`practice/range-queries/` (8 files)

- `problem-1-dynamic-range-sum-queries.cpp`: **Dynamic Range Sum Queries**. Fenwick tree; point assign as a point add of the difference.
- `problem-2-dynamic-range-minimum-queries.cpp`: **Dynamic Range Minimum Queries**. Segment tree, point update, range minimum.
- `problem-3-range-xor-queries.cpp`: **Range Xor Queries**. Prefix XOR, answer p[b] xor p[a-1].
- `problem-4-range-update-queries.cpp`: **Range Update Queries**. Lazy segment tree, range add, point query.
- `problem-5-forest-queries.cpp`: **Forest Queries**. 2D prefix sums with inclusion-exclusion.
- `problem-6-hotel-queries.cpp`: **Hotel Queries**. Max segment tree, descend to the leftmost fit.
- `problem-7-range-updates-and-sums.cpp`: **Range Updates and Sums**. Lazy segment tree with add and assign tags.
- `problem-8-polynomial-queries.cpp`: **Polynomial Queries**. Lazy segment tree storing an arithmetic progression.

`practice/dynamic-programming/` (36 files)

- `understand.cpp`: **(no problem name in the file)**. Per-digit nine's complement, or subtract from a string of ones.
- `problem-1-boredom.cpp`: **Boredom**. DP over values: best[i] = max(best[i-1], best[i-2] + i*cnt[i]).
- `problem-2-consecutive-subsequences.cpp`: **Consecutive Subsequences**. Map dp[x] = dp[x-1] + 1, then rebuild the indices.
- `problem-3-red-green-towers.cpp`: **Red-Green Towers**. Find the max height, then 0/1 subset-count DP over red sums.
- `problem-4-winter-is-here.cpp`: **Winter Is Here**. Count multiples, $c \cdot 2^{c-1}$, inclusion-exclusion.
- `problem-5-frog-1.cpp`: **Frog 1**. Linear DP, jumps of 1 or 2.
- `problem-6-dice-combinations.cpp`: **Dice Combinations**. Dp[i] = sum of dp[i-1..i-6] mod p.
- `problem-7-vlad-and-a-sum-of-sum-of-digits.cpp`: **Vlad and a Sum of Sum of Digits**. Precomputed prefix sums of digit sums.
- `problem-8-minimizing-coins.cpp`: **Minimizing Coins**. Unbounded coin DP, minimum count.
- `problem-9-coin-combinations-i.cpp`: **Coin Combinations I**. Ordered coin-count DP mod p.

- `problem-10-coin-combinations-ii.cpp`: **Coin Combinations II**. Rolling 1D DP, skip or pick each coin.
- `problem-11-removing-digits.cpp`: **Removing Digits**. DP subtracting each nonzero digit.
- `problem-12-grid-paths.cpp`: **Grid Paths**. Grid path-count DP avoiding traps.
- `problem-13-filling-shapes.cpp`: **Filling Shapes**. Closed form $2^{n/2}$, zero for odd n.
- `problem-13-filling-paths-2.cpp`: **Filling Paths 2 (as the file is named)**. Dp[i] = 2 dp[i-2], zero for odd n.
- `problem-14-yet-another-broken-keyboard.cpp`: **Yet Another Broken Keyboard**. Count typable runs, sum of run lengths.
- `problem-16-vitamins.cpp`: **Vitamins**. Cheapest juice per vitamin set, combinations.
- `problem-17-office-keys.cpp`: **Office Keys**. Sorted people and keys, dp[key][person] = min of max time.
- `problem-18-covered-path.cpp`: **Covered Path**. Dp[time][speed] max distance, speed change at most d.
- `problem-19-arrow-paths.cpp`: **Arrow Paths**. BFS on a 2 by n grid with a forced arrow move.
- `problem-20-lecture-sleep.cpp`: **Lecture Sleep**. Prefix and suffix awake sums plus a window of length k.
- `problem-21-frog-2.cpp`: **Frog 2**. Forward DP, jumps of 1 to k.
- `problem-22-vacation.cpp`: **Vacation**. Three-state DP, no activity twice in a row.
- `problem-23-knapsack-1.cpp`: **Knapsack 1**. 0/1 knapsack over weight.
- `problem-24-knapsack-2.cpp`: **Knapsack 2**. Knapsack over value: minimum weight per value.
- `problem-25-lcs.cpp`: **LCS**. LCS table plus backtracking to rebuild the string.
- `problem-26-almost-identity-permutation.cpp`: **Almost Identity Permutation**. Sum of C(n, i) times derangements(i) for i up to 4.
- `problem-28-increasing-subsequence.cpp`: **Increasing Subsequence**. Patience-sorting LIS with lower_bound.
- `problem-29-static-range-sum-queries.cpp`: **Static Range Sum Queries**. Prefix sums.
- `problem-36-pushing-balls.cpp`: **Pushing Balls**. Each 1 needs an all-ones path to the top or the left.
- `problem-37-jellyfish-and-mex.cpp`: **Jellyfish and MEX**. DP from the mex down: dp[j] = min(dp[i] + i*cnt[j]).
- `problem-38-flowers.cpp`: **Flowers**. Ways[i] = ways[i-1] + ways[i-k], prefix sums.
- `problem-39-sleeping-schedule.cpp`: **Sleeping Schedule**. Dp[i][j]: j sleeps taken an hour early.
- `problem-49-python-indentation.cpp`: **Python Indentation**. Dp[line][indent] with suffix-sum transitions.
- `problem-50-yarik-and-array.cpp`: **Yarik and Array**. Kadane, restarting when parity repeats.
- `problem-51-shashlik-and-cooking.cpp`: **Shashlik and Cooking**. Try each start position, step 2k+1, keep the shortest.

practice/graphs/ (65 files)

- `problem-1-kefa-and-park.cpp`: **Kefa and Park**. Recursive tree DFS tracking consecutive-cat streak, counting valid leaves.
- `problem-2-graph-without-long-directed-path.cpp`: **Graph Without Long Directed Path**. Recursive DFS 2-colouring; orient each edge by its endpoint's colour.
- `problem-3-path-queries.cpp`: **Path Queries**. Offline queries sorted by weight; DSU size-based pair counting.
- `problem-4-beautiful-graph.cpp`: **Beautiful Graph**. Recursive DFS bipartite check per component; product of $2^{c_0} + 2^{c_1}$.
- `problem-5-carousel.cpp`: **Carousel**. Cycle of differing neighbours; DFS colouring with 1/2, conflicts use 3.
- `problem-6-cyclic-components.cpp`: **Cyclic Components**. Recursive DFS components; counts those where every degree is 2.
- `problem-7-number-of-simple-paths.cpp`: **Number of Simple Paths**. DFS finds the unique cycle; hanging-tree sizes; complementary counting.
- `problem-8-building-roads.cpp`: **Building Roads**. Recursive DFS component representatives chained as new edges.
- `problem-9-building-teams.cpp`: **Building Teams**. Recursive DFS bipartite 2-colouring over all components.
- `problem-10-number-of-shortest-paths.cpp`: **Number of Shortest Paths**. BFS from node 1 count-

ing shortest paths mod MOD.
- `problem-11-lost-tree.cpp`: **Lost Tree**. Interactive: query root, then the smaller depth-parity class.
- `problem-12-flight-routes-check.cpp`: **Flight Routes Check**. Iterative DFS from node 1 on the graph and on its reverse.
- `problem-13-mahmoud-and-ehab-and-the-bipartiteness.cpp`: **Mahmoud and Ehab and the Bipartiteness**. Iterative DFS tree 2-colouring; cnt0*cnt1-(n-1).
- `problem-14-solve-the-maze.cpp`: **Solve the Maze**. Wall cells next to B, BFS from exit, check reachability.
- `problem-15-police-stations.cpp`: **Police Stations**. Multi-source BFS; edges outside the BFS forest are output.
- `problem-16-web-of-lies.cpp`: **Web of Lies**. Counts nodes with no larger neighbour under updates (no traversal).
- `problem-17-among-us.cpp`: **Among Us**. BFS 2-colouring with same/different constraints; max side per component.
- `problem-18-cover-it.cpp`: **Cover It!**. Iterative DFS 2-colouring; output the smaller class.
- `problem-19-belted-rooms.cpp`: **Belted Rooms**. String scan of $<$ $>$ - characters (no graph).
- `problem-20-forever-winter.cpp`: **Forever Winter**. Degree counting; degree frequencies give the two values.
- `problem-21-white-black-balanced-subtrees.cpp`: **White-Black Balanced Subtrees**. Iterative post-order DFS for subtree balance.
- `problem-22-strongly-connected-city.cpp`: **Strongly Connected City**. Checks four border directions against two patterns (no graph).
- `problem-24-secret-passwords.cpp`: **Secret Passwords**. String-to-letter bipartite graph; count components by DFS.
- `problem-25-hide-and-seek.cpp`: **Hide and Seek**. First/last occurrence arrays, counting adjacent pairs (no graph).
- `problem-26-hongcow-builds-a-nation.cpp`: **Hongcow Builds a Nation**. DFS component sizes from special nodes; complete-graph edges minus m.
- `problem-27-ring-road.cpp`: **Ring Road**. Greedy edge orientation; min(cost, total minus cost).
- `problem-27-ring-road-2.cpp`: **Ring Road (2)**. Recursive DFS around the ring summing reversed-edge costs.
- `problem-28-wormhole-sort.cpp`: **Wormhole Sort**. Binary search on min width plus template DSU connectivity.
- `problem-29-li-hua-and-the-maze.cpp`: **Li Hua and the Maze**. Corner/border case analysis printing 2, 3 or 4 (no graph).
- `problem-30-destroying-bridges.cpp`: **Destroying Bridges**. Closed form comparing k with n-1 (no graph).
- `problem-31-metro.cpp`: **Metro**. Linear scan of two track arrays (no graph).
- `problem-32-protect-sheep.cpp`: **Protect Sheep**. Grid scan: dog next to every sheep, fail on adjacent wolf.
- `problem-33-proving-equivalences.cpp`: **Proving Equivalences**. Template Kosaraju SCC; max of zero in/out-degree counts.
- `problem-34-queen.cpp`: **Queen**. Children lists; per-node child flag check.
- `problem-35-journey.cpp`: **Journey**. Recursive DFS: expected length of the random journey to a leaf.
- `problem-36-sum-in-the-tree.cpp`: **Sum in the Tree**. DFS filling unknown sums with min over children.
- `problem-37-badge.cpp`: **Badge**. Functional graph simulated from every start, $O(n^2)$.
- `problem-38-cooperative-game.cpp`: **Cooperative Game**. Interactive two-speed pointer meeting (Floyd style).
- `problem-39-counting-rooms.cpp`: **Counting Rooms**. Grid flood fill with BFS, counting components.
- `problem-40-planet-cycles.cpp`: **Planet Cycles**. Functional graph: cycle detection plus reverse-edge depths.
- `problem-41-tree-diameter.cpp`: **Tree Diameter**. Two DFS passes: heights, then best two child heights.
- `problem-42-it-restaurants.cpp`: **IT Restaurants**. Subtree sizes, then a bitset subset sum of the parts around each vertex.
- `problem-43-anjis-binary-tree.cpp`: **Anji's Binary Tree**. DFS on a binary tree counting direction changes to a leaf.
- `problem-44-vertical-paths.cpp`: **Vertical Paths**. Post-order DFS splitting the tree into leaf-ended

paths.
- `problem-45-send-the-fool-further-easy.cpp`: **Send the Fool Further (easy)**. DFS for max weighted root-to-leaf distance.
- `problem-46-useful-decomposition.cpp`: **Useful Decomposition**. Degree check, then DFS collects leaves.
- `problem-47-add-on-a-tree.cpp`: **Add on a Tree**. Degree check: any degree-2 node means No.
- `problem-48-tree-jumps.cpp`: **Tree Jumps**. DP by depth level with per-level totals mod MOD.
- `problem-50-productive-meeting.cpp`: **Productive Meeting**. Greedy multiset pairing of the two largest.
- `problem-51-nearest-opposite-polarity.cpp`: **Nearest Opposite Polarity**. Multi-source BFS on reversed jump edges, per parity.
- `problem-52-company-queries-1.cpp`: **Company Queries I**. Binary lifting k-th ancestor.
- `problem-53-company-queries-2.cpp`: **Company Queries II**. Binary lifting LCA.
- `problem-54-distance-queries.cpp`: **Distance Queries**. Binary lifting LCA, distance from depths.
- `problem-56-new-year-tree.cpp`: **New Year Tree**. Euler tour plus lazy segment tree of colour bitmasks, popcount.
- `problem-57-path-queries.cpp`: **Path Queries**. Euler tour with +val/-val plus Fenwick prefix sums.
- `problem-58-sloth-naptime.cpp`: **Sloth Naptime**. Binary lifting LCA plus k-th ancestor jumps.
- `problem-59-static-range-minimum-queries.cpp`: **Static Range Minimum Queries**. Sparse table RMQ, O(1) query (not a graph).
- `problem-60-dzy-loves-chemistry.cpp`: **DZY Loves Chemistry**. DFS component count c; answer 2^{n-c}.
- `problem-61-graph-composition.cpp`: **Graph Composition**. Two DSUs with union by size.
- `problem-62-social-network.cpp`: **Social Network**. DSU by size; sum of the largest components.
- `problem-63-split-into-two-sets.cpp`: **Split Into Two Sets**. Degree/self-loop check, then DFS bipartite check.
- `problem-64-round-dance.cpp`: **Round Dance**. DFS on set-adjacency components, cycles vs paths.
- `problem-65-misha-grisha-and-underground.cpp`: **Misha, Grisha and the Underground**. Binary lifting LCA; max overlap over three endpoint choices.
- `problem-66-knight-tournament.cpp`: **Knight Tournament**. Ordered set of alive indices, erasing each range except x.
- `problem-67-cycle-free-flow.cpp`: **Cycle Free Flow**. Binary lifting LCA with min edge weight on the path.

practice/number-theory/ (48 files)

- `problem-1-maximum-gcd.cpp`: **Maximum GCD**. Closed form: print n/2.
- `problem-2-ehab-and-gcd.cpp`: **EhAb AnD gCd**. Closed form: print 1 and n-1.
- `problem-3-counting-divisors.cpp`: **Counting Divisors**. Trial-division factorisation, product of (exponent+1).
- `problem-4-cat-cycles.cpp`: **Cat Cycle**. Parity case split, modular position with collision offset.
- `problem-5-new-year-and-the-sphere-transmission.cpp`: **New Year and the Sphere Transmission**. Enumerate divisors g, arithmetic-series sum per g, dedupe.
- `problem-6-math.cpp`: **Math**. Prime factorisation, max exponent rounded to a power of two.
- `problem-7-meaningless-operations.cpp`: **Meaningless Operations**. All-ones mask above the top bit; else largest proper divisor.
- `problem-8-floor-and-mod.cpp`: **Floor and Mod**. Square-root decomposition over quotients n/b.
- `problem-9-coprime.cpp`: **Coprime**. Last index per value up to 1000, all value pairs with gcd 1.
- `problem-10-divisors-of-two-integers.cpp`: **Divisors of Two Integers**. Max is x; remove its divisors once; max of the rest is y.
- `problem-11-enlarge-gcd.cpp`: **Enlarge GCD**. Divide by the gcd, smallest-prime-factor sieve, most common prime.
- `problem-12-remainders-game.cpp`: **Remainders Game**. Running gcd(k, lcm(curr, c_i)) == k.
- `problem-13-digits.cpp`: **Digits**. Case analysis on n and d divisibility (1, 3, 5, 7, 9).
- `problem-14-collatz-conjecture.cpp`: **Collatz Conjecture**. Jump to the next multiple of y, then k mod (y-1).
- `problem-15-insane-problem.cpp`: **Insane Problem**. Enumerate powers k^n, intersect the x range per power.
- `problem-16-number-game.cpp`: **Number Game**. Game-theory case split: odd, power of two, n/2

prime.
- `problem-17-maximum-multiple-sum.cpp`: **Maximum Multiple Sum**. Brute force over x, sum of multiples up to n.
- `problem-18-fireworks.cpp`: **Fireworks**. Closed form m/a + m/b + 2.
- `problem-19-di-visible-confusion.cpp`: **Di-visible Confusion**. For each a_i, find some j in [2, i+1] that does not divide it.
- `problem-20-gcd-sequence.cpp`: **GCD Sequence**. Adjacent-gcd array; try deleting one of three candidates.
- `problem-21-polycarp-and-div-3.cpp`: **Polycarp and Div 3**. DP on prefix digit sum mod 3, last index per residue.
- `problem-22-coprime-subsequences.cpp`: **Coprime Subsequences**. Count multiples, 2^c-1, inclusion-exclusion from the top down.
- `problem-23-two-divisors.cpp`: **Two Divisors**. Smallest-prime-factor sieve; first prime against the rest.
- `problem-24-congruence-equation.cpp`: **Congruence Equation**. Loop over the period p-1 with a modular inverse, count periods.
- `problem-25-design-tutorial-learn-from-math.cpp`: **Design Tutorial: Learn from Math**. Sieve; move the n/2 split outward until both are composite.
- `problem-26-divisibility-ofdifferences.cpp`: **Divisibility of Differences**. Group by residue mod m, output k with the same residue.
- `problem-27-diverse-matrix.cpp`: **Diverse Matrix**. Constructive: cell (x, y) = x(y+r), single row or column special case.
- `problem-28-santas-bot.cpp`: **Santa's Bot**. Expected probability with modular inverses and frequency counts.
- `problem-29-tile-painting.cpp`: **Tile Painting**. Factorise: prime power gives p, prime gives n, else 1.
- `problem-30-orac-and-lcm.cpp`: **Orac and LCM**. Suffix gcd; gcd over lcm(a_i, suffix gcd).
- `problem-31-hilberts-hotel.cpp`: **Hilbert's Hotel**. (i + a_i mod n) mod n must be a permutation.
- `problem-32-same-gcds.cpp`: **Same GCDs**. Euler phi of m/gcd(a, m) by trial division.
- `problem-33-nearly-shortest-repeating-substring.cpp`: **Nearly Shortest Repeating Substring**. Each divisor length, mismatches against the prefix and suffix block.
- `problem-34-rectangles.cpp`: **Rectangles**. Per row and column, count 0s and 1s, sum subsets, minus n*m.
- `problem-35-archers.cpp`: **Archers**. Geometric series p1 / (1 - (1-p1)(1-p2)).
- `problem-36-monotonic-renumeration.cpp`: **Monotonic Renumeration**. Merge overlapping first/last spans; answer 2^{s-1} for s segments.
- `problem-37-wet-shark-and-flowers.cpp`: **Wet Shark and Flowers**. Per neighbour pair, probability of a multiple of p, times 2000.
- `problem-38-ilya-and-escalator.cpp`: **Ilya and Escalator**. Probability DP over people and seconds.
- `problem-39-little-pony-and-the-expected-maximum.cpp`: **Little Pony and the Expected Maximum**. $E[\max] = \sum_i i\,((i/m)^n - ((i-1)/m)^n)$.
- `problem-41-makes-and-the-product.cpp`: **Makes and the Product**. Sort; case analysis on run lengths of the smallest values.
- `problem-41-makes-and-the-product-2.cpp`: **Makes and the Product (second version)**. Sort; counts of the three smallest, combinatorial cases.
- `problem-42-parking-lot.cpp`: **Parking Lot**. Closed-form counting with powers of 4.
- `problem-43-total-appeal-of-a-string.cpp`: **Total Appeal of a String**. Per character, subtract the substrings that skip it.
- `problem-44-expected-median.cpp`: **Expected Median**. Sum of C(zeros, i)C(ones, k-i) for i below (k+1)/2.
- `problem-45-jury-meeting.cpp`: **Jury Meeting**. N! minus bad arrangements via nCr and factorials.
- `problem-46-dp-problem-1.cpp`: **DP Problem 1**. 1D DP with steps of 1 or 2.
- `problem-47-dp-problem-2.cpp`: **DP Problem 2**. 2D additive grid propagation, Yes if a cell equals t.
- `problem-48-fun.cpp`: **Fun**. Enumerate (a, b) with ab up to n and a+b up to x; count c by formula.

practice/bitmasks/ (8 files)

- `problem-1-co-growing-sequence.cpp`: **Co-growing Sequence**. Running prefix OR; y_i = prefix OR xor x_i.

- `problem-2-game-with-multiset.cpp`: **Game with Multiset**. Counts per power of two, greedy subtraction from the high bit.
- `problem-3-gardener-and-the-array.cpp`: **Gardener and the Array**. Global bit counts; YES if some element's bits all appear twice.
- `problem-5-cat-fox-and-the-lonely-array.cpp`: **Cat, Fox and the Lonely Array**. Binary search on k plus sliding-window per-bit counts.
- `problem-6-data-structures-fan.cpp`: **Data Structures Fan**. Prefix XOR; a range flip XORs both group totals.
- `problem-7-sheikh-easy-version.cpp`: **Sheikh (Easy Version)**. Binary search on length, prefix sum minus prefix XOR.
- `problem-8-bitwise-balancing.cpp`: **Bitwise Balancing**. Closed form a = (d|b) - (d&c), then verify.
- `problem-16-neko-performs-cat-furrier-transformation.cpp`: **Neko Performs Cat Furrier Transformation**. Alternate XOR with $2^k - 1$ and $+1$ until all bits are ones.

practice/prefix-sums/ (13 files)

- `problem-a-kuriyama-mirais-stones.cpp`: **Kuriyama Mirai's Stones**. Two prefix-sum arrays, original and sorted.
- `problem-b-vika-and-squares.cpp`: **Vika and Squares**. Min times n plus the longest cyclic run above the min.
- `problem-c-balanced-tunnel.cpp`: **Balanced Tunnel**. Map entry order to exit order, count overtakes.
- `problem-d-karen-and-coffee.cpp`: **Karen and Coffee**. Difference array, threshold k, prefix count.
- `problem-e-greg-and-array.cpp`: **Greg and Array**. Two nested difference arrays (operation counts, then values).
- `problem-f-star-sky.cpp`: **Star Sky**. 2D prefix sums per starting brightness; a star shines $(s+t)$ mod $(c+1)$.
- `problem-g-little-girl-and-maximum-sum.cpp`: **Little Girl and Maximum Sum**. Difference-array frequencies, sort both, pair greedily.
- `problem-h-approximating-a-constant-range.cpp`: **Approximating a Constant Range**. Sliding window with an ordered map (max - min at most 1).
- `problem-i-good-subarrays.cpp`: **Good Subarrays**. Prefix sum of (digit - 1), count equal prefixes in a map.
- `problem-j-encrypting-messages.cpp`: **Encrypting Messages**. Difference array of key additions over a window, mod c.
- `problem-k-and-its-non-zero.cpp`: **AND, its Non-Zero**. Per-bit set count on [l, r] by closed form, n minus the max.
- `problem-l-subsequence-hate.cpp`: **Subsequence Hate**. Prefix 0/1 counts, best split for 0*1* or 1*0*.
- `problem-m-least-prefix-sum.cpp`: **Least Prefix Sum**. Greedy sign flips with a max-heap left of m and a min-heap right.

practice/two-pointers/ (7 files)

- `problem-a-news-about-credit.cpp`: **News About Credit**. Sort by capacity, greedy assignment of informants.
- `problem-b-kuroni-and-simple-strings.cpp`: **Kuroni and Simple Strings**. Two pointers from both ends pairing (with).
- `problem-c-lecture-sleep.cpp`: **Lecture Sleep**. Prefix sums of all and awake values, best k-window.
- `problem-d-money-trees.cpp`: **Money Trees**. Sliding window reset when h[j-1] % h[j] != 0, sum at most k.
- `problem-e-mathematical-problem.cpp`: **Mathematical Problem**. Brute force over the merge position.
- `problem-f-air-conditioner.cpp`: **Air Conditioner**. Propagate the reachable temperature interval over time.
- `problem-g-rudolf-and-k-bridges.cpp`: **Rudolf and k Bridges**. Monotonic-deque DP per row, then a window of k rows.

practice/mixed/ (101 files)

- problem-a-lazy-caterer-sequence.cpp: **Lazy Caterer Sequence**. Closed form n(n+1)/2+1.
- problem-b-pacifist-frogs.cpp: **Pacifist Frogs**. Brute force divisibility counting; collect min-count indices.
- problem-c-spyke-chatting.cpp: **Spyke Chatting**. Read matrix rows as strings; index 2*(y-1) per message.
- problem-1-wonderful-gloves.cpp: **Wonderful Gloves**. Greedy: sum per-pair max, add top K-1 mins, +1.
- problem-2-wonderful-city.cpp: **Wonderful City**. Two-state row DP, transpose matrix, repeat for columns.
- problem-3-chaya-calendar.cpp: **Chaya Calendar**. Greedy: jump to next strictly-greater multiple per element.
- problem-4-remove-two-letters.cpp: **Remove Two Letters**. Count i where s[i-1]==s[i+1], subtract from N-1.
- problem-5-quick-sort.cpp: **Quick Sort**. Count increasing 1,2,3.. subsequence; ceil(remaining/K).
- problem-6-reverse-card-easy-version.cpp: **Reverse Card (Easy Version)**. Closed-form sum over b of (N+b)/(b*b), minus 1.
- problem-7-gorilla-and-permutation.cpp: **Gorilla and Permutation**. Construct: N down to M+1, then 1..M.
- problem-8-robot-program.cpp: **Robot Program**. Simulate first hit of 0, then count full cycles.
- problem-9-cake-assembly-line.cpp: **Cake Assembly Line**. Intersect allowed shift intervals (max lower, min upper).
- problem-10-serval-and-string-theory.cpp: **Serval and String Theory**. Compare with reverse; else need S>0 and >1 distinct chars.
- problem-11-combination-lock.cpp: **Combination Lock**. Output N..1 if N odd, else -1.
- problem-12-having-been-a-treasurer-in-the-past-i-help-goblins-decieve.cpp: **Having Been a Treasurer... Goblins Deceive**. Count '-' and '_'; split dashes in half, multiply.
- problem-13-asuna-and-the-mosquitoes.cpp: **Asuna and the Mosquitoes**. Parity count: max if one parity, else sum-odd+1.
- problem-14-burenka-plays-with-fractions.cpp: **Burenka Plays with Fractions**. Cross-multiply; answer 0/1/2 by divisibility.
- problem-15-binary-inversions.cpp: **Binary Inversions**. Three passes: flip first 0, flip last 1, none.
- problem-16-removing-smallest-multiples.cpp: **Removing Smallest Multiples**. Sieve-like multiples walk, stop at kept element.
- problem-17-rooks-defenders.cpp: **Rooks Defenders**. Sets of free rows/cols + counts; lower_bound range check.
- problem-18-floor-or-ceil.cpp: **Floor or Ceil**. Simulate two orderings (ceil-first vs floor-first) with early exits.
- problem-19-koxia-and-permutation.cpp: **Koxia and Permutation**. Construct by alternating large blocks and small values via map.
- problem-20-trust-nobody.cpp: **Trust Nobody**. Brute-force liar count c with count(x>c)==c.
- problem-21-jellyfish-and-green-apple.cpp: **Jellyfish and Green Apple**. Reduce fraction; power-of-two check; popcount*M-N.
- problem-22-bewitching-stargazer.cpp: **Bewitching Stargazer**. Binary halving recursion summarised as bit-weighted sum.
- problem-23-sets-and-union.cpp: **Sets and Union**. Coordinate compress + bitset unions excluding each element.
- problem-24-particles.cpp: **Particles**. Max of positive sums on odd vs even indices.
- problem-25-treasure-hunt.cpp: **Treasure Hunt**. Modular check a % (x+y) >= x.
- problem-26-kamilka-and-the-sheep.cpp: **Kamilka and the Sheep**. Sort; output max-min.
- problem-27-lady-bug.cpp: **Lady Bug**. Parity-position counting of 1s vs 0s across two strings.
- problem-28-serval-and-final-mex.cpp: **Serval and Final MEX**. Simulation: merge adjacent zeros via vector erase.
- problem-28-serval-and-final-mex-2.cpp: **Serval and Final MEX (v2)**. Constructive: split halves containing 0, then whole.
- problem-29-disappearing-permutation.cpp: **Disappearing Permutation**. Cycle walk with map/visited per query.
- problem-29-disappearing-permutation-2.cpp: **Disappearing Permutation (v2)**. Cycle marking

with set, print set size per query.
- `problem-30-you-soared-afar-with-grace.cpp`: **You Soared Afar With Grace**. Constructive swaps pairing a/b mirror positions.
- `problem-39-team-training.cpp`: **Team Training**. Sort desc; greedy grouping until size*min >= X.
- `problem-40-large-array-and-segments.cpp`: **Large Array and Segments**. Count full copies, then scan partial copy prefix.
- `problem-41-creating-keys-for-storages-has-become-my-main-skill.cpp`: **Creating Keys for Storages...**. Bitwise-OR constructive: prefix 0..i submask of X, pad X.
- `problem-42-limited-repainting.cpp`: **Limited Repainting**. Binary search on threshold; count B-segments.
- `problem-43-olympiad-date.cpp`: **Olympiad Date**. Multiset of required digits (0,1,2,3,5) via map countdown.
- `problem-44-cloudberry-jam.cpp`: **Cloudberry Jam**. Closed form via double arithmetic (N*3/0.75/2).
- `problem-45-min-gcd.cpp`: **Min GCD**. GCD of multiples of min (excluding min) equals min.
- `problem-46.cpp`: **(no problem name in the file)**. Interval intersection of feasible x; fixed sum consistency check.
- `problem-47.cpp`: **(no problem name in the file)**. Prefix/suffix greedy subsequence matching, try inserting one value.
- `problem-48-simple-palindrome.cpp`: **Simple Palindrome**. Distribute N evenly over 5 vowels, print grouped.
- `problem-49-distinct-character-queries.cpp`: **Distinct Character Queries**. Per-char ordered sets of positions; lower_bound per query.
- `problem-50-thor.cpp`: **Thor**. Notification log with per-app cursor and global read pointer.
- `problem-51-matrix-exploration.cpp`: **Matrix Exploration**. Multi-source BFS on grid, sum distances.
- `problem-51-sort.cpp`: **Sort**. 26-letter prefix counts; sum abs diff / 2 per query.
- `problem-52-unfair-game.cpp`: **Unfair Game**. Sum non-negatives; parity-based greedy pick of sorted negatives.
- `problem-53-simple-permutation.cpp`: **Simple Permutation**. Linear sieve; centre permutation around prime near N/2.
- `problem-54-binary-matrix.cpp`: **Binary Matrix**. Greedy parity fixes: flip cells for odd rows/-columns.
- `problem-55-the-play-never-ends.cpp`: **The Play Never Ends**. Check N % 3 == 1.
- `problem-56-perfecto.cpp`: **Perfecto**. Identity permutation, swap where prefix sum is perfect square.
- `problem-57-mex-game-1.cpp`: **MEX Game 1**. Frequency map; stop at second singleton or missing value.
- `problem-58-forbidden-difference.cpp`: **Forbidden Difference**. Group by value mod d; take/skip DP on chains.
- `problem-59-turtle-and-mex-problem-easy-version.cpp`: **Turtle and MEX Problem (Easy)**. Second-MEX per sequence; closed-form sum to M.
- `problem-60-binary-coloing.cpp`: **Binary Colouring**. Non-adjacent signed-digit binary rewrite of N.
- `problem-61-mystic-permutation.cpp`: **Mystic Permutation**. Identity perm; swap with neighbour on fixed-point collision.
- `problem-62-to-zero.cpp`: **To Zero**. Parity math: one odd step then ceil by even K.
- `problem-63-xor-and-triangle.cpp`: **XOR and Triangle**. Brute-force y over two-bit masks; triangle inequality.
- `problem-64-lol-lovers.cpp`: **LOL Lovers**. Prefix counts against suffix counts (suffix recounted for each i).
- `problem-65-waste-sorting.cpp`: **Waste Sorting**. Greedy capacity subtraction for typed/mixed items.
- `problem-66-competitive-programmer.cpp`: **Competitive Programmer**. Divisibility by 60 via digit sum, zero, even count.
- `problem-67-william-the-vigilant.cpp`: **William the Vigilant**. Maintain "abc" count; local recount around updated index.
- `problem-68-diamond-miner.cpp`: **Diamond Miner**. Abs coords, sort both axes, pair and sum hypotenuses.
- `problem-69-elas-fitness-and-the-luxury-number.cpp`: **Ela's Fitness and the Luxury Number**. Integer sqrt via binary search; count k*k, k(k+1), k(k+2).

- `problem-70-quingshan-loves-strings-2.cpp`: **Qingshan Loves Strings 2**. Constructive: insert "01" at mismatching end, cap 300 ops.
- `problem-71-game-on-tree-easy.cpp`: **Game on Tree (Easy)**. Binary lifting LCA; leaf-to-start distance parity.
- `problem-72-mad-mad-sum.cpp`: **Mad MAD Sum**. Apply MAD transform twice, then weighted closed-form sum.
- `problem-73-elections.cpp`: **Elections**. Prefix sums with winner index; per-candidate removal count.
- `problem-74-the-great-julya-calendar.cpp`: **The Great Julya Calendar**. Digit-subtract DP to 2e6 plus memoised recursion beyond.
- `problem-75-new-skateboard.cpp`: **New Skateboard**. Divisible-by-4 via last digit and last two digits.
- `problem-76-constanzes-machine.cpp`: **Constanze's Machine**. Fibonacci-style DP on uu/nn runs; zero if w/m.
- `problem-77-basketball-exercise.cpp`: **Basketball Exercise**. Rolling 3-state DP (row1, row2, skip).
- `problem-78-kirei-attacks-the-estate.cpp`: **Kirei Attacks the Estate**. Tree DFS with alternating-sum max (pos/neg carry).
- `problem-79-come-a-little-closer.cpp`: **Come a Little Closer**. Bounding box; try removing each extreme point.
- `problem-80-cs-academy-consecutive-subsequence.cpp`: **Consecutive Subsequence (CS Academy)**. Value-indexed DP with one-skip state.
- `problem-81-cs-academy-divisor-clique.cpp`: **Divisor Clique (CS Academy)**. Sort; $O(N^2)$ longest divisibility-chain DP.
- `problem-82-max-even-subarray.cpp`: **Max Even Subarray**. Prefix sums; min prefix per index parity.
- `problem-83-recursive-string.cpp`: **Recursive String**. Tribonacci lengths; recursive K-th character descent.
- `problem-84-display-the-number.cpp`: **Display the Number**. Greedy: leading 7 if odd, then 1s.
- `problem-85-deadly-laser.cpp`: **Deadly Laser**. Check both boundary paths blocked; else N+M-2.
- `problem-86-username.cpp`: **Username**. String parse: strip trailing digits after last letter.
- `problem-87-build-the-permutation.cpp`: **Build the Permutation**. Constructive peaks/valleys from set extremes.
- `problem-88-interview.cpp`: **Interview**. Interactive binary search with prefix sums, flushed queries.
- `problem-89.cpp`: **(no problem name in the file)**. Map value to first/last index; greedy interval union.
- `problem-90-flower-boy.cpp`: **Flower Boy**. Prefix/suffix greedy matching; try inserting each b[i].
- `problem-91-tenzing-and-balls.cpp`: **Tenzing and Balls**. DP with per-value best bucket.
- `problem-93-cs-academy-to-front-to-back.cpp`: **To Front to Back (CS Academy)**. Longest consecutive-value increasing-position run; output moves.
- `problem-94-cs-academy-and-closure.cpp`: **AND Closure (CS Academy)**. Superset-AND DP over values descending.
- `problem-95-letter-home.cpp`: **Letter Home**. Case analysis on S vs min/max of sorted array.
- `problem-96-above-the-clouds.cpp`: **Above the Clouds**. Detect repeated middle char.
- `problem-97-those-who-are-with-us.cpp`: **Those Who Are With Us**. Check if one row+column covers all max cells.
- `problem-98-cs-academy-bst-fixed-height.cpp`: **BST Fixed Height (CS Academy)**. DP over count and height bound; difference of bounds.
- `problem-99.cpp`: **(no problem name in the file)**. Brute force 1..100 for a strictly closer point.
- `problem-100-mex-and-array.cpp`: **MEX and Array**. Contribution counting: subarrays containing i.
- `problem-101-paranoid-string.cpp`: **Paranoid String**. Count substrings ending at i by adjacent-difference rule.
- `problem-102-limited-swaps-cs-academy.cpp`: **Limited Swaps (CS Academy)**. Segment tree (max+count, k-th) greedy lexicographic extraction.
- `problem-102.cpp`: **(no problem name in the file)**. Brute-force triangle counting via adjacency list + set of sets.
- `problem-103.cpp`: **(no problem name in the file)**. At-most-K-transactions stock DP; unlimited case if K>=N/2.

practice/cs-academy/ (8 files)

- problem-1-.cpp: **(no problem name in the file)**. Sliding-window max on differences (monotonic stack).
- problem-2-adjacent-vowels.cpp: **Adjacent Vowels**. Count adjacent vowel pairs.
- problem-3-fill-the-glasses.cpp: **Fill the Glasses**. Sort, sum K smallest, ceil divide by 100.
- problem-4-donkey-paradox.cpp: **Donkey Paradox**. Brute force grid cells with equal Manhattan distances.
- problem-5-matrix-exploration.cpp: **Matrix Exploration**. Multi-source BFS on grid, sum distances.
- problem-7-black-shapes.cpp: **Black Shapes**. DFS component labelling; try filling each 0 cell.
- problem-8-check-dfs.cpp: **Check DFS**. Sort adjacency by target order, run DFS, compare sequences.
- problem-9-connect-the-graph.cpp: **Connect the Graph**. DFS components + non-tree edges reused to link leaders.

practice/atcoder/ (3 files)

- problem-1-atcoder-language.cpp: **AtCoder Language**. String lookup via nested ternary.
- problem-2-get-min.cpp: **Get Min**. Multiset insert / extract-min.
- problem-4-kings-summit-2.cpp: **King's Summit 2**. Bounding box; ceil(max(dx,dy)/2).

contests/codeforces/ (30 files)

- problem-b-cost-of-the-array.cpp: **Cost of the Array**. Case analysis on k == n and the first non-1 position.
- problem-c-customer-service.cpp: **Customer Service**. Trailing ones per row, sort, greedy MEX.
- problem-a-2.cpp: **Round 1008 A**. Compare the mean with x.
- problem-a.cpp: **Round 1017 A**. Concatenate first letters.
- problem-b.cpp: **Round 1017 B**. Closed-form shift of an interval.
- problem-c.cpp: **Round 1017 C**. Collect seen values, prepend the missing one.
- problem-d.cpp: **Round 1017 D**. Run-length encode both strings, compare runs.
- problem-e.cpp: **Round 1017 E**. Per-bit counts, best XOR sum over pivots.
- problem-f.cpp: **Round 1017 F**. Constructive grid colouring by modular index.
- problem-g.cpp: **Round 1017 G**. Deque plus a reverse flag, O(1) sum updates.
- problem-a-lever.cpp: **Lever**. Sum of positive a - b, plus one.
- problem-b-alternating-series.cpp: **Alternating Series**. Constructive -1, 3 pattern.
- problem-c-make-it-equal.cpp: **Make It Equal**. Multiset of min(r, k-r) residues compared.
- problem-d-arboris-contractio.cpp: **Arboris Contractio**. Leaves minus the most leaves sharing one neighbour.
- problem-e-adjacent-xor.cpp: **Adjacent XOR**. Greedy left-to-right fix with backward propagation.
- problem-a-min-mex-max.cpp: **Min-Mex-Max**. All values other than -1 equal and nonzero.
- problem-b-hamiiid-haaamid-...- -hamid.cpp: **Hamiiid, Haaamid... Hamid**. One wall on each side, best escape distance.
- problem-c-later-trip-shopping.cpp: **Round 1041 C**. Sort intervals, sum lengths, add twice the min gap.
- problem-d-root-was-built-by-love-,-broken-by-destiny.cpp: **Root was Built by Love, Broken by Destiny**. Caterpillar check, 2 or 4 times a product of factorials.
- problem-a.cpp: **Round 1043 A**. String prepend and append simulation.
- problem-b.cpp: **Round 1043 B**. Divisors of n of the form $10^k + 1$.
- problem-c.cpp: **Round 1043 C**. Bounding-box centre, max Chebyshev distance.
- problem-d.cpp: **Round 1043 D**. Base-3 digits, greedy splitting to spend k.
- problem-a.cpp: **Round 1065 A**. Brute-force count of even splits.
- problem-b.cpp: **Round 1065 B**. Fill -1 with the endpoints or 0.
- problem-d.cpp: **Round 1065 D**. Prefix-min scan tracking an invalid max.
- problem-e.cpp: **Round 1065 E**. Group by mod 2 and 3, emit triples.
- problem-f.cpp: **Round 1065 F**. D's scan plus edge building, DFS components, link them.
- problem-c1.cpp: **Round 1065 C1**. Total XOR plus parity of the last differing index.
- problem-c2.cpp: **Round 1065 C2**. Top bit of the total XOR, last index with that bit.

contests/atcoder/ (11 files)

- problem-a.cpp: **ABC 396 A**. Scan for three equal consecutive values.
- problem-b.cpp: **ABC 396 B**. Stack simulation.
- problem-c.cpp: **ABC 396 C**. Sort descending, prefix sums; best white count k paired with at least k black balls.
- problem-a.cpp: **ARC 194 A**. DP: $d_i = \max(d_{i-1} + a_i,\ d_{i-2})$, since dropped steps form even blocks.
- problem-b.cpp: **ARC 194 B**. Sum of |p_i - i|.
- problem-2.cpp: **ABC 398 B**. Frequency map, full-house check.
- problem-1.cpp: **ARC 195 A**. Earliest against latest subsequence match.
- problem-2.cpp: **ARC 195 B**. Pairwise frequency-sum map against the max element.
- problem-6.cpp: **ABC 399 F**. Prefix sums plus binomial expansion of $(P_r - P_l)^k$.
- problem-6-2.cpp: **ABC 399 F (second version)**. The same method, heavily commented.
- problem-6-3.cpp: **ABC 399 F (third version)**. The same method, binomials from Pascal's triangle.

course-cs334/class-contests/ (8 files)

- problem-b-rakhshs-revival.cpp: **Rakhsh's Revival**. Greedy scan of zero runs of length m.
- problem-c-ternary-string.cpp: **Ternary String**. Last positions of 1, 2, 3, minimum window.
- problem-a-1d-array.cpp: **1D Array**. Greedy: jump k from each next B.
- problem-a-and-there-were-k.cpp: **And Then There Were K**. $2^{\lfloor \log_2 n \rfloor} - 1$.
- problem-b-and-reconstruction.cpp: **AND Reconstruction**. B[i] = a[i-1] | a[i], then verify.
- problem-c-shohag-loves-xor-easy-version.cpp: **Shohag Loves XOR (Easy)**. Brute force over $y \le \min(m, 2x)$ with a divisibility check.
- problem-c.cpp: **Lecture Sleep**. Two prefix sums, window of length k.
- problem-a.cpp: **Contest 11 A**. Multinomial n! / prod(cnt!) mod p.

course-cs334/labs/ (7 files)

- problem-b-ab-flipping.cpp: **AB Flipping**. First A, last B, difference.
- problem-d-longest-strike.cpp: **Longest Strike**. Sort, values with frequency at least k, longest run.
- problem-e-sakurakos-field-trip.cpp: **Sakurako's Field Trip**. Swap each mirrored pair when a neighbour matches, then count duplicates.
- problem-a-xorwice.cpp: **XORwice**. Print a xor b.
- problem-b-roof-construction.cpp: **Roof Construction**. Place 0 at the highest power of two up to n-1.
- problem-d-and-an-infinite-sequence.cpp: **...And an Infinite Sequence**. OR of every value in $[n - m, n + m]$, bit by bit.
- problem-e-vlad-and-division.cpp: **Vlad and Division**. Pair x with its complement $2^{31} - 1 - x$ via a map.

course-cs334/problem-sets/ (8 files)

- problem-e-seating-in-a-bus.cpp: **Seating in a Bus**. Each new seat must have an occupied neighbour.
- problem-f-black-and-white-strip.cpp: **Black and White Strip**. Sliding-window count of B.
- problem-g-maximum-sum.cpp: **Maximum Sum**. Sort plus prefix sums, try i removals of the max.
- problem-h-dora-and-search.cpp: **Dora and Search**. Two pointers shrinking min and max endpoints.
- problem-a-raising-bacteria.cpp: **Raising Bacteria**. Count set bits by halving.
- problem-b-fedor-and-new-game.cpp: **Fedor and New Game**. XOR, count ones through a bitset string.
- problem-c-pter-and-combination-lock.cpp: **Petr and Combination Lock**. Bitmask brute force over the signs.
- problem-d-preparing-olympiad.cpp: **Preparing Olympiad**. Bitmask subset enumeration with sum and range checks.

from-other-sources/cses-dp-priyansh-agarwal/ (20 files)

- 1_dice_combinations.cpp: **Dice Combinations**. Forward and backward 1D DP.

- `1_dice_combinations_optimized_space.cpp`: **Dice Combinations**. Sliding window of the last six states.
- `2_minimizing_coins.cpp`: **Minimizing Coins**. Unbounded coin minimum DP.
- `3_coin_combinations_1.cpp`: **Coin Combinations I**. Ordered coin-count DP.
- `4_coin_combinations_2.cpp`: **Coin Combinations II**. 2D dp[i][sum], skip or pick.
- `4_coin_combinations_2_optimized_space.cpp`: **Coin Combinations II**. Rolling next and current state.
- `5_removing_digits_dp.cpp`: **Removing Digits**. DP over digits.
- `5_removing_digits_greedy.cpp`: **Removing Digits**. Greedy: subtract the largest digit.
- `6_grid_paths.cpp`: **Grid Paths**. Backward path-count DP.
- `6_grid_paths_optimized_space.cpp`: **Grid Paths**. Rolling rows, backward.
- `7_book_shop.cpp`: **Book Shop**. 2D 0/1 knapsack.
- `7_book_shop_optimized_space.cpp`: **Book Shop**. Rolling 1D knapsack.
- `8_array_description.cpp`: **Array Description**. Dp[i][last value], transitions from k-1 to k+1.
- `9_counting_towers_1.cpp`: **Counting Towers**. Two-state DP recomputed per test.
- `9_counting_towers_2.cpp`: **Counting Towers**. Two-state DP precomputed once.
- `10_edit_distance.cpp`: **Edit Distance**. Levenshtein 2D DP.
- `10_edit_distance_space_optimized.cpp`: **Edit Distance**. Rolling rows, shorter string as columns.
- `11_rectangle_cutting.cpp`: **Rectangle Cutting**. Dp[a][b], minimum over every cut.
- `12_money_sums.cpp`: **Money Sums**. 2D boolean subset-sum DP.
- `14_two_sets_2.cpp`: **Two Sets II**. Subset-count DP times the inverse of 2.

`from-other-sources/contest-problems/` (10 files)

- `card-game.cpp`: **Card Game**. 3D counting DP by turn pattern.
- `cs-academy-and-closure.cpp`: **AND Closure**. Sum-over-subsets DP over supermasks.
- `cs-academy-subarray-removal.cpp`: **Subarray Removal**. Kadane prefix and suffix arrays combined around the gap.
- `it-restaurants.cpp`: **IT Restaurants**. Subtree sizes, rerooting, subset sum per node.
- `photoshoot-for-gorillas.cpp`: **Photoshoot for Gorillas**. Cell coverage counts sorted, paired with sorted heights.
- `the-intriguing-obsession.cpp`: **The Intriguing Obsession**. Matching-count DP for each pair of colours, multiplied.
- `unforgivable-curse-easy-version.cpp`: **Unforgivable Curse (Easy)**. BFS over swaps for small n, else multiset equality.
- `atcoder-beginner-contest-400-problem-g.cpp`: **ABC 400 G**. Sort by max, prefix and suffix DP over 12 states.
- `codeforces-round-1016-shorten-the-array.cpp`: **Shorten the Array**. Binary trie max-XOR plus two pointers.
- `codeforces-round-1017-problem-h.cpp`: **Round 1017 H**. Position lists per value, jump to the next divisor occurrence.